

Practical-1

Name: Muskan Chaurasia

PRN:- 202501050014

1.1.1 Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: where: is the mass of the object (in kilograms). is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity
```

```
= float(input())
```

```
momentum = mass*velocity print(f"{momentum:.2f}kgm/s")
```

The screenshot shows a web-based code execution interface. At the top, there's a header with 'Anywhere' on the left, a user ID '202501050014@mitaoe.ac.in' and 'Support' link in the center, and a 'Logout' button on the right. Below the header, the main area is titled 'calculate...' and shows execution statistics: 'Average time 0.011 s (11.50 ms)' and 'Maximum time 0.015 s (15.00 ms)'. It also displays '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. The test cases are listed below:

Test case	Time	Expected output	Actual output
Test case 1	15 ms	5.0 10.0 50.00kgm/s	5.0 10.0 50.00kgm/s
Test case 2	14 ms	2.3 4.8 11.04kgm/s	2.3 4.8 11.04kgm/s

At the bottom, there are tabs for 'Terminal' and 'Test cases'.

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$$1 \leq \leq 999$$

Input Format:

- The input consists of a single integer .

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Code:

```
n=int(input()) if
```

```
0<=n and n<=9:
```

```
    print(n*n)        elif
```

```
10<=n and n<=99:
```

```
    print(f"{n**(1/2):.2f}") elif
```

```
100<=n and n<=999:
```

```
    print(f"{n**(1/3):.2f}") else:
```

```
    print("Invalid")
```

Anywhere ▾ 202501050014@mitaoe.ac.in ▾ Support Logout

Explorer

Average time 0.010 s 9.71 ms Maximum time 0.016 s 16.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 16 ms Debug

Expected output	Actual output
9	9
81	81

Test case 2 9 ms Debug

Expected output	Actual output
166	166
5.50	5.50

Test case 3 8 ms Debug

Expected output	Actual output
24	24
4.90	4.90

Test case 4 8 ms Debug

Expected output	Actual output
3456	3456
Invalid	Invalid

Debugger

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD
- The second line contains the second date in the format YYYY-MM-DD
- An integer representing the number of days between the given dates.

Output Format:

Code: from datetime import datetime

```
date1=input().strip() date2=input().strip()
```

```
d1=datetime.strptime(date1,"%Y-%m-%d")
```

```
d2=datetime.strptime(date2,"%Y-%m-%d")
```

```
difference = d2-d1 print(difference.days)
```

[Home](#)
[Learn Anywhere](#)

202501050014@mitaoe.ac.in
Support
Logout

noOfDay...

Average time
0.022 s
22.50 ms

Maximum time
0.058 s
58.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 58 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 8 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-07-11	2014-07-11
9	9

Terminal
Test cases

< Prev
Reset
Submit
Next >

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
num = int(input()) reversed_num=
int(str(num)[::-1])
print(reversed_num)
```

[Home](#)
[Learn Anywhere](#)

202501050014@mitaoe.ac.in
Support
Logout

Average time

0.011 s

11.40 ms

Maximum time

0.014 s

14.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

10 ms

Debug

Expected output	Actual output
5367	5367
7635	7635

Test case 2

10 ms

Debug

Expected output	Actual output
8132	8132
231	231

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format: **<number> x**

<multiplier> = <result> Code:

```
num = int(input())
```

```
i in range(1,11):
```

```
    print(f"{num} x {i} = {num * i}")
```

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

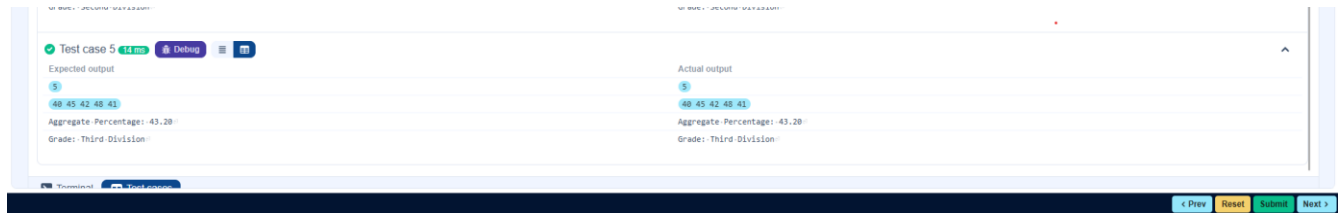
- "Fail".

Code:

```
def cal(marks, total_courses):  
    if any(mark < 40 for mark in marks):  
        return "Fail"  
    total_marks = sum(marks)  
    aggregate_percentage=(total_marks/(total_courses *100))*100  
    if aggregate_percentage >75: grade="Distinction"  
    elif aggregate_percentage >=60: grade= "First Division"  
    elif aggregate_percentage >=50: grade= "Second Division"  
    elif aggregate_percentage >=40: grade= "Third Division"  
    return (aggregate_percentage,grade)  
num_courses= int(input())  
marks= list(map(int,input().split()))  
result= cal(marks,num_courses)  
if result == "Fail":  
    print("Fail")  
else:  
    aggregate_percentage,grade=result  
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")  
    print(f"Grade: {grade}")
```

The screenshot displays the CODETANTRA IDE interface. At the top, the header includes the logo, navigation links (Home, Learn Anywhere), a user email (202501050014@mitaoe.ac.in), and a Logout button. The main workspace shows the code from the previous block. Below the code editor, a test results panel is visible, indicating that 5 out of 5 shown test cases passed and 5 out of 5 hidden test cases passed. The panel lists four test cases with their respective inputs, expected outputs, and actual outputs. Test case 1 and 2 passed, while Test case 3 failed and Test case 4 passed.

Test Case	Input	Expected Output	Actual Output	Status
Test case 1	55 65 78 89	Aggregate Percentage: 71.75 Grade: First Division	Aggregate Percentage: 71.75 Grade: First Division	Passed
Test case 2	56 78 97 86 93	Aggregate Percentage: 82.00 Grade: Distinction	Aggregate Percentage: 82.00 Grade: Distinction	Passed
Test case 3	99 85 43 97 34	Fail	Fail	Failed
Test case 4	55 54 53	Aggregate Percentage: 54.00 Grade: Second Division	Aggregate Percentage: 54.00 Grade: Second Division	Passed



1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first `n` terms of the Fibonacci series.

Input Format:

- A single integer `n` representing the number of terms to generate.

Output Format:

- A single line containing the first `n` terms of the Fibonacci sequence, separated by spaces.

Code: def

fibonacci(n):

if n == 1:

return 0

elif n == 2:

return 1

else:

return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input())

for i in range(1, n + 1):

print(fibonacci(i), end=" ")

[Home](#)
[Learn Anywhere](#)

202501050014@mitaoe.ac.in
[Support](#)
[Logout](#)

Average time

0.031 s

31.00 ms

Maximum time

0.038 s

38.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

38 ms

Debug

Expected output

5

0 1 1 2 3

Actual output

5

0 1 1 2 3

Test case 2

36 ms

Debug

Expected output

15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output

15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

< Prev

Reset

Submit

Next >

1.2.3. Pattern -1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern. **Output Format**
- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
n = int(input())
for i in range(1, n+1):
    print('* ' * i)
```

[Home](#)
[Learn Anywhere](#)

202501050014@mitaoe.ac.in
Support
Logout

Average time
0.010 s
10.33 ms

Maximum time
0.012 s
12.00 ms

2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed

Test case 1 **12 ms**
Debug

Expected output
5

Actual output
5

Test case 2 **11 ms**
Debug

Expected output
4

Actual output
4

Terminal
Test cases

< Prev
Reset
Submit
Next >

1.2.4. Pattern -2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n = int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Explorer

Average time
0.009 s
8.75 msMaximum time
0.010 s
10.00 ms**2 out of 2 shown test case(s) passed**
2 out of 2 hidden test case(s) passed

>

☰

✓ Test case 1 **9 ms** [Debug](#) [Test cases](#)

Expected output

5

1 - 1

1 - 2 - 1

1 - 2 - 3 - 1

1 - 2 - 3 - 4 - 1

1 - 2 - 3 - 4 - 5 - 1

Actual output

5

1 - 1

1 - 2 - 1

1 - 2 - 3 - 1

1 - 2 - 3 - 4 - 1

1 - 2 - 3 - 4 - 5 - 1

✓ Test case 2 **8 ms** [Debug](#) [Test cases](#)

Expected output

8

1 - 1

1 - 2 - 1

1 - 2 - 3 - 1

1 - 2 - 3 - 4 - 1

1 - 2 - 3 - 4 - 5 - 1

1 - 2 - 3 - 4 - 5 - 6 - 1

1 - 2 - 3 - 4 - 5 - 6 - 7 - 1

1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 1

Actual output

8

1 - 1

1 - 2 - 1

1 - 2 - 3 - 1

1 - 2 - 3 - 4 - 1

1 - 2 - 3 - 4 - 5 - 1

1 - 2 - 3 - 4 - 5 - 6 - 1

1 - 2 - 3 - 4 - 5 - 6 - 7 - 1

1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 1

Terminal

Test cases

Debugger

< Prev Reset Submit Next >